Caso 5: Edificios Inteligentes y Energía

Documentación técnica completa del proyecto de Machine Learning para predicción y clasificación de eficiencia energética en edificios

Python · scikit-learn Dataset UCI ENB2012 Regresión Lineal (L1/L2)

Regresión Logística (L1/L2) CRISP-DM 55 celdas · 7 secciones

CONTENIDO DEL DOCUMENTO

0. Contexto del negocio y objetivos
1. El dataset: ENB2012
2. Sección 1 – Preparación del entorno
3. Sección 2 – Cargado de datos
4. Sección 3 – Análisis Exploratorio (EDA)
5. Sección 4 – División de los datos
6. Sección 5 – Pipeline de preprocesamiento
7. Sección 6 – Regresión lineal
8. Sección 7 – Regresión logística
9. Resumen y conclusiones finales

0 Contexto del negocio y objetivos

La motivación del problema y las dos tareas de Machine Learning

🏢 ¿Por qué predecir la energía de un edificio?

En la construcción de edificios sostenibles —en especial los que buscan certificaciones ambientales como **LEED**— conocer el consumo energético *antes de construir* es fundamental por tres razones concretas:

Razón	Impacto
Ahorro económico	Menos gasto en facturas de luz y gas durante toda la vida del edificio. Un error de diseño cuesta años de sobreconsumo.
Dimensionamiento de equipos	Saber exactamente qué capacidad necesita el sistema de calefacción o aire acondicionado. Un equipo sobredimensionado se compra de más; uno subdimensionado no alcanza.
Cumplimiento normativo	Muchas legislaciones exigen límites de consumo energético. Predecirlo permite rediseñar antes de empezar la obra.

La energía que necesita un edificio para mantener una temperatura confortable depende directamente de su **forma y diseño físico**: qué tan compacto es, cuántas ventanas tiene, hacia qué lado está orientado, cuántos pisos tiene, etc. Estas características son exactamente las variables del dataset.

🎯 Las dos tareas del proyecto

TAREA 1 – REGRESIÓN

Predecir el **valor numérico exacto** de la *carga de calefacción* (`Heating_Load` / Y1), medida en kWh/m². Esto permite a los diseñadores saber con precisión cuánta energía consumirá el edificio.

📐 **Algoritmos:** Regresión Lineal (OLS), Ridge (L2), Lasso (L1)

TAREA 2 – CLASIFICACIÓN

Etiquetar cada diseño como **Eficiente (1)** o **No Eficiente (0)** según si su `Heating_Load` supera un umbral definido. Un diseño "no eficiente" se descarta por consumir demasiada energía.

🏷️ **Algoritmos:** Regresión Logística (sin reg., L2, L1)

1 El dataset: ENB2012

Origen, estructura y descripción de cada variable

📦 Origen y características generales

768	8	2	0
Registros (edificios simulados)	Variables predictoras (X)	Variables objetivo (Y1, Y2)	Valores nulos

UCI
Fuente del dataset

El dataset fue generado mediante simulaciones del software **Ecotect**, variando sistemáticamente 8 parámetros de diseño arquitectónico en distintas combinaciones. Todos los edificios simulados tienen el mismo volumen total, pero distintas proporciones y configuraciones de superficie y ventanas. Las dos variables objetivo son la **carga de calefacción** (Y1) y la **carga de refrigeración** (Y2), aunque en este proyecto solo se utiliza Y1.

📋 Variables del dataset (nombres originales → nombres del proyecto)

Original	Nombre en proyecto	Tipo	Descripción	Rango
X1	`Relative_Compactness`	**Numérica**	Compacidad relativa del edificio. Relación entre volumen y superficie.	0.62 –

X2	`Surface_Area`	**Numérica**	Área de la superficie exterior total en m².	514 – 8
X3	`Wall_Area`	**Numérica**	Área total de paredes en m².	245 –
X4	`Roof_Area`	**Numérica**	Área del techo en m².	110 – 2
X5	`Overall_Height`	**Numérica**	Altura total del edificio. Solo dos valores: 3.5 m (1 piso) o 7 m (2 pisos).	3.5 / 7.
X6	`Orientation`	**Categórica**	Orientación principal del edificio. Norte / Este / Sur / Oeste.	4 cate
X7	`Glazing_Area`	**Numérica**	Porcentaje del Surface_Area cubierto por ventanas (0 = sin ventanas, 0.4 = 40%).	0.0 – 0
X8	`Glazing_Area_Distribution`	**Categórica**	Distribución de las ventanas: Sin acristalamiento / Uniforme / Norte / Este / Sur / Oeste.	6 cate
			Carga de	

Y1	`Heating_Load`	**Target reg.**	calefacción en kWh/m². Variable objetivo principal.	6.01 –
Y2	`Cooling_Load`	**Ignorada**	Carga de refrigeración. No se usa en este proyecto.	10.9 –
—	`Eficiente`	**Target clf.**	Variable binaria creada en el proyecto. 1 si Heating_Load ≤ mediana, 0 si no.	0 / 1

2 Sección 1 – Preparación del entorno

Celdas 1–7 | Dependencias, hardware y configuración visual

📦 1.1 – Gestión de librerías e importaciones

Todas las importaciones del proyecto se centralizan dentro de la función `import_libraries()` . Esta función declara las variables como `global` para que estén disponibles en cualquier celda posterior sin necesidad de importar de nuevo. El flujo de ejecución es a prueba de fallos:

Intento 1:

ejecutar `import_libraries()` directamente.

Si falla (ImportError):

buscar `requirements.txt` en la carpeta padre y ejecutar `pip install -r` silenciosamente.

Intento 2:

re-ejecutar `import_libraries()` tras la instalación. Si falla de nuevo, mostrar error crítico con el motivo.

Librería	Propósito en el proyecto
`numpy`	Operaciones numéricas (sqrt, random seed), cálculo de sesgo, matrices
`pandas`	Carga del Excel, manipulación del DataFrame, renombrado de columnas
`matplotlib` / `seaborn`	Todos los gráficos: histogramas, boxplots, heatmaps, scatter, barras, ROC
`scikit-learn`	Pipeline, ColumnTransformer, StandardScaler, OneHotEncoder, modelos y métricas
`psutil`	Leer RAM y CPU del sistema para verificar requisitos mínimos
`zipfile` / `urllib`	Descargar y extraer el dataset desde la UCI automáticamente

🔍 1.2 – Verificación de requisitos mínimos del entorno

La función `verificar_requisitos_minimos()` compara el entorno actual con los requisitos del entorno original de desarrollo. El objetivo es detectar incompatibilidades antes de ejecutar el proyecto completo.

Requisito	Mínimo	Severidad si falla
Python	≥ 3.12.12	❌ Bloqueo: posibles errores de sintaxis

RAM	≥ 12.67 GB	❌ Bloqueo: riesgo de Out-of-Memory
CPU (núcleos lógicos)	≥ 2	❌ Bloqueo: ejecución muy lenta
Sistema Operativo	Linux (recomendado)	⚠️ Advertencia: puede funcionar igual

🎨 1.3 – Configuración global de visualización

Se configuran los parámetros globales de `matplotlib` para que todos los gráficos del proyecto tengan el mismo estilo tipográfico, asegurando consistencia visual y reproducibilidad:

```
plt.rc('font', size=12)
plt.rc('axes', labelsize=14, titlesize=14)
plt.rc('legend', fontsize=12)
plt.rc('xtick', labelsize=10)
plt.rc('ytick', labelsize=10)
```

3 Sección 2 – Cargado de datos

Celdas 8–12 | Descarga, extracción, carga, renombrado y variable de clasificación

🌐 Descarga automática del dataset

Se establece la semilla global `SEED = 42` y se define la función `descargar_datos_energy_efficiency()`. Esta función garantiza que todos los usuarios trabajen con exactamente los mismos datos sin descargas manuales:

Tras cargar el DataFrame se verifican tres **assertions de integridad**: que tenga exactamente 768 filas y 10 columnas, que sea un DataFrame de pandas, y que la columna `Y1` (objetivo) exista.

🏷️ Renombrado y decodificación de columnas

Las columnas genéricas `X1…X8, Y1, Y2` se reemplazan por nombres descriptivos. Adicionalmente, los valores numéricos de las columnas categóricas se decodifican a texto:

Orientación (X6)

```
{2: 'Norte', 3: 'Este',
 4: 'Sur',   5: 'Oeste'}
```

Distribución de acristalamiento (X8)

```
{0: 'Sin acristalamiento',
 1: 'Uniforme',
 2: 'Norte', 3: 'Este',
 4: 'Sur',   5: 'Oeste'}
```

✨ 2.1 – Creación de la variable objetivo de clasificación (`Eficiente`)

Se crea la variable binaria `Eficiente` que permite la segunda tarea del proyecto (clasificación). Se transforma `Heating_Load` usando la **mediana como umbral**:

```
UMBRAL_EFICIENCIA = energyEfficiency['Heating_Load'].median()  # ≈ 18.95
energyEfficiency['Eficiente'] = (
    energyEfficiency['Heating_Load'] <= UMBRAL_EFICIENCIA
).astype(int)
```

⚖️ ¿Por qué la mediana como umbral?

La mediana divide el dataset exactamente en dos mitades iguales (50/50), lo que produce clases perfectamente balanceadas. Esto evita el sesgo del modelo hacia la clase mayoritaria y elimina la necesidad de técnicas adicionales como SMOTE o `class_weight='balanced'`. Además, la mediana es un estadístico intrínseco de los datos: es reproducible y no requiere ninguna decisión externa arbitraria.

✅ `Eficiente = 1`
Heating_Load ≤ 18.95
384 edificios (50%)

❌ `Eficiente = 0`
Heating_Load > 18.95
384 edificios (50%)

4 Sección 3 – Análisis Exploratorio de Datos (EDA)

Celdas 13–38 | Inspección, distribuciones, outliers, escalas, correlaciones y multicolinealidad

🔎 Inspección inicial del DataFrame

Se ejecuta `.info()` para conocer tipos de datos y valores nulos, y `.describe().T` para las estadísticas descriptivas (media, desviación estándar, percentiles). Luego se muestran los **valores únicos** de cada variable, lo que revela información clave sobre la naturaleza del dataset:

Hallazgo	Variable	Implicación
Solo 12 valores únicos	Surface_Area	Dataset de simulaciones sistemáticas, no datos continuos reales
Solo dos valores (3.5 y 7.0)	Overall_Height	Representa "1 piso" vs "2 pisos", es casi binaria
4 categorías textuales	Orientation	Ya decodificada de números a nombres

Glazing_Area va de 0 a 0.4	Glazing_Area	Es un *ratio* del Surface_Area, no m² directos (0.1 = 10% de ventanas)
6 categorías	Glazing_Area_Distribution	Indica hacia dónde apuntan las ventanas

📊 Análisis univariado – `analisis_univariado_completo()`

Para cada variable numérica se genera un **histograma con curva KDE** y se calcula el *sesgo (skewness)*. El diagnóstico automático clasifica la distribución:

Condición	Diagnóstico	Acción recomendada
`|sesgo| > 1`	⚠️ Sesgo alto	Considerar transformación logarítmica o escalador robusto
`0.5 < |sesgo| ≤ 1`	🟡 Sesgo moderado	Vigilar; puede requerir tratamiento
`|sesgo| ≤ 0.5`	✅ Simétrico	No requiere transformación

Para las variables categóricas (`Orientation` , `Glazing_Area_Distribution`) se generan **gráficos de barras de frecuencia** con el conteo exacto de cada clase.

> ✅ **Conclusión:** Ninguna variable numérica tiene sesgo alto (máximo: 0.53, moderado). Las clases categóricas están balanceadas. No se requieren transformaciones especiales.

🎯 Análisis de la variable objetivo `Heating_Load`

Se analiza por separado la distribución de `Heating_Load` con el mismo esquema de histograma + KDE + diagnóstico de sesgo, para decidir si necesita transformación antes de modelar.

✅ **Conclusión:** No es necesario aplicar `log10` ni ninguna otra transformación al target. Su distribución es aceptablemente simétrica para regresión lineal, sin colas extremas que distorsionen el entrenamiento.

✨ Balance de clases de `Eficiente`

Inmediatamente después del análisis de `Heating_Load`, se verifica el balance de clases de la variable binaria derivada de ella. Se usa un `countplot` con anotaciones de conteo y porcentaje, el mismo estilo que las variables categóricas del EDA.

✅ **Resultado:** 384 registros por clase (50% / 50%). Balance perfecto. No se necesitan técnicas de balanceo como SMOTE ni ajuste de `class_weight`.

📦 Análisis de Outliers – `analisis_outliers()`

Para cada variable numérica se dibuja un **boxplot horizontal** y se aplica el método del **Rango Intercuartílico (IQR)** para detectar outliers matemáticamente:

```
Q1  = df[col].quantile(0.25)
Q3  = df[col].quantile(0.75)
IQR = Q3 - Q1
limite_inferior = Q1 - 1.5 * IQR
limite_superior = Q3 + 1.5 * IQR
outliers = df[(df[col] < limite_inferior) | (df[col] > limite_superior)]
```

Se reporta el porcentaje de outliers de cada variable. El factor `1.5×IQR` es el criterio estándar de Tukey; valores fuera de ese rango se consideran atípicos.

✅ **Conclusión:** Cero outliers detectados en ninguna variable. No se necesita ningún tratamiento de valores atípicos en el pipeline.

📏 Análisis bivariado de escalas – `analizar_escalas()`

Se comparan las escalas de todas las variables numéricas en un **boxplot horizontal unificado**. La tabla de resumen muestra min, max, media, std y rango total de cada variable.

Variable	Rango aprox.
Surface_Area	514 – 808 (rango ≈ 294)
Wall_Area	245 – 416 (rango ≈ 171)
Roof_Area	110 – 220 (rango ≈ 110)
Overall_Height	3.5 – 7.0 (rango = 3.5)
Glazing_Area	0.0 – 0.4 (rango = 0.4)
Relative_Compactness	0.62 – 0.98 (rango ≈ 0.36)

⚖️ **Decisión: usar StandardScaler en el pipeline**

Las diferencias de escala son enormes: Surface_Area llega a 808, mientras Glazing_Area máximo es 0.4. Sin escalado, los algoritmos de gradiente (como la Regresión Logística) convergen más lento, y los algoritmos con regularización penalizan de forma desproporcionada a las variables con valores grandes. El **StandardScaler** normaliza cada variable a media 0 y desviación estándar 1, eliminando el efecto de escala.

🎯 Dispersión vs target – `analizar_dispersion_target()`

Para cada variable numérica se genera un `sns.regplot` contra `Heating_Load` : puntos semi-transparentes (alpha=0.5) más una línea de regresión lineal simple en rojo. Se calcula la **correlación de Pearson** y se imprime un diagnóstico automático:

|Pearson|	Diagnóstico

≥ 0.7	✅ Relación fuerte – muy útil para el modelo
0.3 – 0.7	🟡 Relación moderada – aporta información
< 0.3	⚠️ Relación débil – posible nube sin patrón

> ✅ **Conclusión:** Ninguna variable fue descartada por baja correlación con el target.

🔥 Multicolinealidad – `analizar_multicolinealidad()`

Se calcula la matriz de correlación de Pearson entre todas las variables numéricas (incluido el target) y se visualiza como un **heatmap triangular** (solo el triángulo inferior para evitar redundancia). Un escáner automático alerta cuando `|r| ≥ 0.8` entre dos predictores.

> 🚨 **Se detectaron dos pares con multicolinealidad severa:**
>
> - `Relative_Compactness` ↔ `Surface_Area` : correlación muy alta (inversa)
> - `Roof_Area` ↔ `Overall_Height` : correlación muy alta (inversa)

> ⚖️ **Decisión: eliminar Relative_Compactness y Roof_Area**
>
> Cuando dos variables están altamente correlacionadas entre sí, aportan información redundante y producen coeficientes inestables en regresión lineal. Se debe eliminar una de cada par. El criterio de selección fue conservar la variable con *menor correlación con el target* (ya que la otra aporta más información independiente):
>
> - Par 1: se elimina `Relative_Compactness` , se conserva `Surface_Area` .
> - Par 2: se elimina `Roof_Area` , se conserva `Overall_Height` .

5

Sección 4 – División de los datos

Celda 40 | Train/test split para regresión y clasificación

✂️ Limpieza y partición

Primero se eliminan las dos variables descartadas por multicolinealidad, luego se separan features y targets, y finalmente se realiza la partición:

```
# Eliminar variables con multicolinealidad severa
df_limpio = energyEfficiency.drop(columns=['Roof_Area', 'Relative_Compactness'])

# --- REGRESIÓN ---
X = df_limpio.drop(columns=['Heating_Load'])
y = df_limpio['Heating_Load']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# --- CLASIFICACIÓN (mismo X, distinto y) ---
y_clf = energyEfficiency.loc[X.index, 'Eficiente']
_, _, y_clf_train, y_clf_test = train_test_split(
    X, y_clf, test_size=0.2, random_state=42)
```

614	154	7
Filas de entrenamiento (80%)	Filas de prueba (20%)	Variables en X

🔗 **Coherencia entre tareas:** al usar `random_state=42` en ambos splits con el mismo `X`, se garantiza que la fila 47 de `X_train` corresponde exactamente a la misma fila en `y_clf_train`. Las particiones son idénticas en índices, permitiendo comparaciones directas entre regresión y clasificación.

Las 7 variables que componen `X`:

Surface_Area **Wall_Area** **Overall_Height** **Glazing_Area** **Orientation** **Glazing_Area_Distribution** **Cooling_Load (ignorada por el pipeline)**

6 Sección 5 – Pipeline de preprocesamiento

Celda 42 | ColumnTransformer con StandardScaler y OneHotEncoder

🏭 Construcción del preprocesador

Se construye un `ColumnTransformer` que aplica transformaciones diferentes según el tipo de columna. Este objeto se convierte en el **primer paso de todos los Pipelines** del proyecto (tanto de regresión como de clasificación):

```
preprocesador = ColumnTransformer(
    transformers=[
        ('numericas',   StandardScaler(),
         ['Surface_Area', 'Wall_Area', 'Overall_Height', 'Glazing_Area']),

        ('categoricas', OneHotEncoder(drop='first', sparse_output=False),
         ['Orientation', 'Glazing_Area_Distribution'])
    ],
    remainder='drop'  # Descarta Cooling_Load, Eficiente y cualquier columna extra
)
```

Transformación	Columnas	Motivo
StandardScaler	Surface_Area, Wall_Area, Overall_Height, Glazing_Area	Escalas muy diferentes detectadas en el EDA. Normaliza a μ=0, σ=1.
OneHotEncoder `drop='first'`	Orientation, Glazing_Area_Distribution	Convierte categorías a variables binarias. `drop='first'` elimina una categoría por variable para evitar la trampa de variables dummy (multicolinealidad perfecta entre dummies).
`remainder='drop'`	Cooling_Load, Eficiente, etc.	Descarta silenciosamente cualquier columna no especificada en los transformadores.

🛡️ **¿Por qué usar Pipeline y no transformar directamente?**
Con `Pipeline`, el `preprocesador` se entrena (`.fit()`) *únicamente* con los datos de entrenamiento. Cuando se transforma el test set, se usan los parámetros aprendidos en train (media y std del StandardScaler). Esto evita el **data leakage**: que información del test contamine el entrenamiento.

Dimensiones de salida del preprocesador:

- 4 columnas numéricas escaladas
- 3 dummies de Orientation (4 categorías – 1)
- 5 dummies de Glazing_Area_Distribution (6 categorías – 1)
- **Total: 12 columnas** de entrada al algoritmo

7 Sección 6 – Regresión: predecir Heating_Load

Celdas 43–46 | LinearRegression, Ridge y Lasso comparados con RMSE, MAE y R^2

🧮 Los tres modelos de regresión

Modelo	Regularización	Penalización	Cuándo es mejor
LinearRegression	Ninguna (OLS puro)	—	Cuando no hay sobreajuste. Sirve como línea base de comparación.
Ridge (L2) `alpha=1.0`	L2 (norma cuadrática)	$\lambda \Sigma \beta_i^2$	Cuando hay multicolinealidad residual. Encoge coeficientes sin anularlos.
Lasso (L1) `alpha=1.0`	L1 (valor absoluto)	$\lambda \Sigma \lvert\beta_i\rvert$	Cuando se sospecha que algunas variables son irrelevantes. Puede anularlas exactamente.

🔄 Loop de entrenamiento y evaluación

Cada modelo se integra en un Pipeline completo y se entrena/evalúa con el mismo código:

```
for nombre_modelo, modelo in diccionario_modelos.items():

    pipeline_final = Pipeline(steps=[
        ('preprocesamiento', preprocesador),
        ('algoritmo',       modelo)
    ])
    pipeline_final.fit(X_train, y_train)
    predicciones = pipeline_final.predict(X_test)

    rmse = np.sqrt(mean_squared_error(y_test, predicciones))
    mae  = mean_absolute_error(y_test, predicciones)
    r2   = r2_score(y_test, predicciones)

    # Scatter: valores reales vs predichos + línea ideal (roja punteada)
    plt.scatter(y_test, predicciones, alpha=0.6, color='dodgerblue')
    plt.plot([y_test.min(), y_test.max()],
             [y_test.min(), y_test.max()], 'r--', lw=2)
```

RMSE

$\sqrt{(1/n \cdot \Sigma(y-\hat{y})^2)}$

Error en las mismas unidades del target (kWh/m²). Penaliza errores grandes. **Menor = mejor.** Es la métrica principal de ordenación.

MAE

$1/n \cdot \Sigma|y-\hat{y}|$

Error promedio absoluto. Más robusto que RMSE ante outliers. **Menor = mejor.**

R^2

$1 - SS_{res}/SS_{tot}$

Proporción de varianza explicada. 1.0 = perfecto, 0 = modelo nulo. **Mayor = mejor.**

📈 **Gráfico Reales vs Predichos:** si el modelo es perfecto, todos los puntos caen sobre la línea roja punteada (pendiente 1, intercept 0). Los puntos dispersos arriba o abajo de esa línea indican el error de cada predicción.

8 Sección 7 – Clasificación: predecir Eficiente

Celdas 47–54 | LogisticRegression sin regularización, L2 y L1

🧠 ¿Por qué Regresión Logística para clasificación binaria?

La **Regresión Logística** es el algoritmo de clasificación binaria más natural y explicable. En lugar de predecir un valor continuo, modela la *probabilidad* de pertenecer a la clase positiva:

```
P(Eficiente=1 | X) = σ(β₀ + β₁·X₁ + ... + βₙ·Xₙ)
donde σ(z) = 1/(1 + e⁻ᶻ)  ← función sigmoide
```

Si `P ≥ 0.5` → predice clase 1 (Eficiente); si `P < 0.5` → clase 0. Los coeficientes del modelo tienen interpretación en términos de *log-odds*, lo que permite entender el impacto de cada variable en la eficiencia del edificio.

⚙️ Las tres variantes comparadas

Variante	penalty	solver	C	Efecto
Sin regularización	`None`	`lbfgs`	—	Modelo base. Puede sobreajustar si hay muchas variables. Referencia de comparación.
Ridge (L2)	`'l2'`	`lbfgs`	1.0	Encoge todos los coeficientes hacia 0 sin anularlos. Muy robusto ante multicolinealidad

				residual. El más estable en datasets pequeños.
Lasso (L1)	`'l1'`	`liblinear`	1.0	Puede anular coeficientes exactamente a 0. Funciona como selector automático de variables. Requiere solver `liblinear` (lbfgs no soporta L1).

> 💡 **El parámetro C** en scikit-learn es la *inversa* de la fuerza de regularización (C = 1/λ). Un C alto significa poca regularización; un C bajo, mucha. Se usó `C=1.0` (valor por defecto) como punto de partida neutral para una comparación justa entre variantes.

🔄 Loop de entrenamiento y métricas de clasificación

Se sigue el mismo patrón del loop de regresión: iterar sobre el diccionario, construir el Pipeline completo (mismo `preprocesador`), entrenar y evaluar. La diferencia clave es que aquí también se obtienen **probabilidades** para calcular el ROC-AUC:

```
pipeline_clf.fit(X_train, y_clf_train)
predicciones_clf = pipeline_clf.predict(X_test)
probabilidades   = pipeline_clf.predict_proba(X_test)[:, 1]  # prob. de clase 1
```

Accuracy

`(TP + TN) / N`

Proporción de predicciones correctas. Funciona bien con clases balanceadas (nuestro caso).

Precision

`TP / (TP + FP)`

De los predichos como Eficiente, ¿cuántos lo son? Penaliza falsos positivos.

Recall

`TP / (TP + FN)`

De los realmente Eficientes, ¿cuántos se detectaron? Penaliza falsos negativos.

F1-Score

`2·P·R / (P+R)`

Media armónica de Precision y Recall. Métrica principal de ordenación en la tabla

de resultados.

ROC-AUC

$\int TPR\ d(FPR)$

Área bajo la curva ROC. Independiente del umbral de corte. 1.0 = perfecto, 0.5 = aleatorio.

📊 Matrices de confusión y curvas ROC

Para cada modelo se genera una **Matriz de Confusión** con `ConfusionMatrixDisplay.from_predictions()`, etiquetando las clases como "No Eficiente" y "Eficiente":

	Pred: No Eficiente	Pred: Eficiente
Real: No Eficiente	TN ✅	FP ❌
Real: Eficiente	FN ❌	TP ✅

Las **Curvas ROC** se generan para los tres modelos en un mismo eje con `RocCurveDisplay.from_estimator()`, permitiendo comparación visual directa. Se añade la línea diagonal punteada (clasificador aleatorio, AUC=0.50) como referencia. Cuanto más cerca del vértice superior-izquierdo, mejor el modelo.

> ✅ Se espera que todos los modelos obtengan **AUC ≥ 0.90**, dado que el dataset es limpio, balanceado y las variables físicas del edificio tienen buena capacidad predictiva sobre su eficiencia energética.

9 Resumen y conclusiones finales

Visión integral del proyecto, decisiones clave y estructura completa

🗺️ Flujo completo del proyecto

UCI ENB2012 (768 filas × 10 col.) → **Renombrar Decodificar** → **Crear Eficiente** (mediana=umbral) → **EDA completo** (6 sub-análisis) →

Eliminar Rel_Compact+Roof → **Train/Test** (80/20 seed=42) → **Pipeline** (StdScaler+OHE)

LinReg / Ridge / Lasso (RMSE · MAE · R^2) + **LogReg sin·L2·L1** (Acc·Prec·Rec·F1·AUC)

⚖️ Registro de decisiones técnicas del proyecto

#	Decisión	Alternativa descartada	Justificación
1	No transformar `Heating_Load` con log10	Aplicar log10	El sesgo era acepta (ninguna variable superó 1.0)
2	No tratar outliers	Winsorización, eliminación	El método IQR no detectó ningún outli en ninguna variable
3	Usar `StandardScaler`	MinMaxScaler, RobustScaler	Las escalas son mu diferentes; StandardScaler es e estándar para regresión/logística
			Relative_Compactn

4	Eliminar `Relative_Compactness`	Eliminar Surface_Area	tenía mayor correlación con Surface_Area → má redundante
5	Eliminar `Roof_Area`	Eliminar Overall_Height	Roof_Area tenía ma correlación con Overall_Height → m redundante
6	`OneHotEncoder(drop='first')`	drop=None	Evitar multicolinealidad perfecta entre dummies (trampa d variables dummy)
7	Mediana como umbral de `Eficiente`	Media, percentil 25/75, valor fijo	Produce 50/50 de balance, reproducib sin parámetros externos
8	`random_state=42` en todo	Distintas semillas	Reproducibilidad to mismos resultados cada ejecución
9	`solver='liblinear'` para Lasso	lbfgs para L1	lbfgs no soporta penalización L1; liblinear es eficiente para datasets pequeños

📁 Estructura de archivos del proyecto

Archivo	Descripción
`Caso5_linear_regression.ipynb`	Notebook principal con las 55 celdas del proyecto

	completo
`datasets/energyEfficiency.zip`	Dataset descargado automáticamente desde la UCI
`datasets/energyEfficiency_data/ENB2012_data.xlsx`	Dataset extraído y listo para leer con pandas
`explicacion_notebook.html`	Documentación del notebook original (secciones 1–6 de regresión)
`informe_regresion_logistica.html`	Informe técnico de cómo se implementó la clasificación
`proyecto_completo.html`	Este documento: visión integral del proyecto completo

📐 Estructura final del notebook (55 celdas)

Celdas	Sección	Contenido	Tipo
0	Encabezado	Contexto del negocio, objetivos y tareas	Markdown
1–7	Sección 1	Imports, verificación de entorno, configuración visual	Código + MD
8–10	Sección 2	Descarga, carga y renombrado del dataset	Código + MD
11–12	Sección 2.1	Creación de la variable binaria `Eficiente`	NUEVO

13–24	Sección 3	EDA: inspección, univariado, Heating_Load	**Código + MD**
25–27	Sección 3	EDA balance de clases de `Eficiente`	**NUEVO**
28–38	Sección 3	EDA: outliers, escalas, dispersión, multicolinealidad	**Código + MD**
39–40	Sección 4	División train/test (regresión + clasificación)	**Código (mod.)**
41–42	Sección 5	Pipeline de preprocesamiento	**Código + MD**
43–46	Sección 6	Regresión: diccionario_modelos, loop, evaluación	**Código + MD**
44	Sección 6	Definición de `diccionario_modelos` (fix del bug)	**NUEVO**
47–54	Sección 7	Clasificación con Regresión Logística (L0, L2, L1), métricas, ROC, conclusiones	**NUEVO**